Course architecture

Advanced Statistics 2027-1 / Probabilistic Machine Learning · Course-level intellectual and curricular authority

This document defines the **intellectual purpose and planned progression** of the course. It is the primary reference when selecting lecture scopes, weighing changes across lectures, and interpreting the relationship between mathematical results and applications. The student syllabus is a concise description of this architecture; approved lecture scopes determine their own detailed teaching commitments. Actual delivery and source coverage are recorded in the material ledger. These authorities should be reconciled when they disagree, not silently overwritten.

What the course is for

The goal is **working competence in probabilistic machine learning**: given a substantive dataset and a scientific or practical question, students should be able to recognize the kind of learning problem, select and use an appropriate model and computational method, assess the result, and explain when another approach would be preferable. They should leave familiar with specific algorithms **as working tools**, not simply with their derivations or names. This is a graduate course in which students already know probability and Bayesian statistics; it develops that background into the ability to tackle new machine-learning problems.

A **coherent probabilistic understanding** is the main intellectual instrument for developing this competence, **not an end in itself**. Models, performance criteria, conditioning, marginalization, optimization and approximation provide a common language for comparing apparently different algorithms. Their mathematical derivations matter: without them, students cannot see what an algorithm computes, why its assumptions help, when it fails, or how it differs from an alternative. Yet coherence must serve the problems. We should not organize a lecture merely to fill in another piece of a beautiful general theory, nor reduce the semester to a catalogue of algorithms and applications.

The course therefore has **two intertwined threads**. One develops the mathematical ability to formulate and compare methods. The other builds a concrete repertoire: particular learning problems, datasets, implementations, performance checks and diagnoses. Every substantive lecture should advance both threads at an appropriate depth. The algorithm or mathematical principle is an **exemplar** of a transferable idea; the example is substantial enough that students learn how to use and question the method, rather than merely recognize its algebra. Practical competence here means credible, guided first use and informed method selection—not unassisted production mastery of every technique after one notebook.

From learning problems to mathematical formulations

A **learning task** specifies what a system should accomplish; its **experience** is the data, feedback, or interaction available. The **structure and scale** of that experience matter. Its success must be judged by a **performance requirement**: what predictions, distributions, representations or decisions must be good, and how good is good enough? The same dataset can support prediction, generation, discovery or action, each with different legitimate criteria.

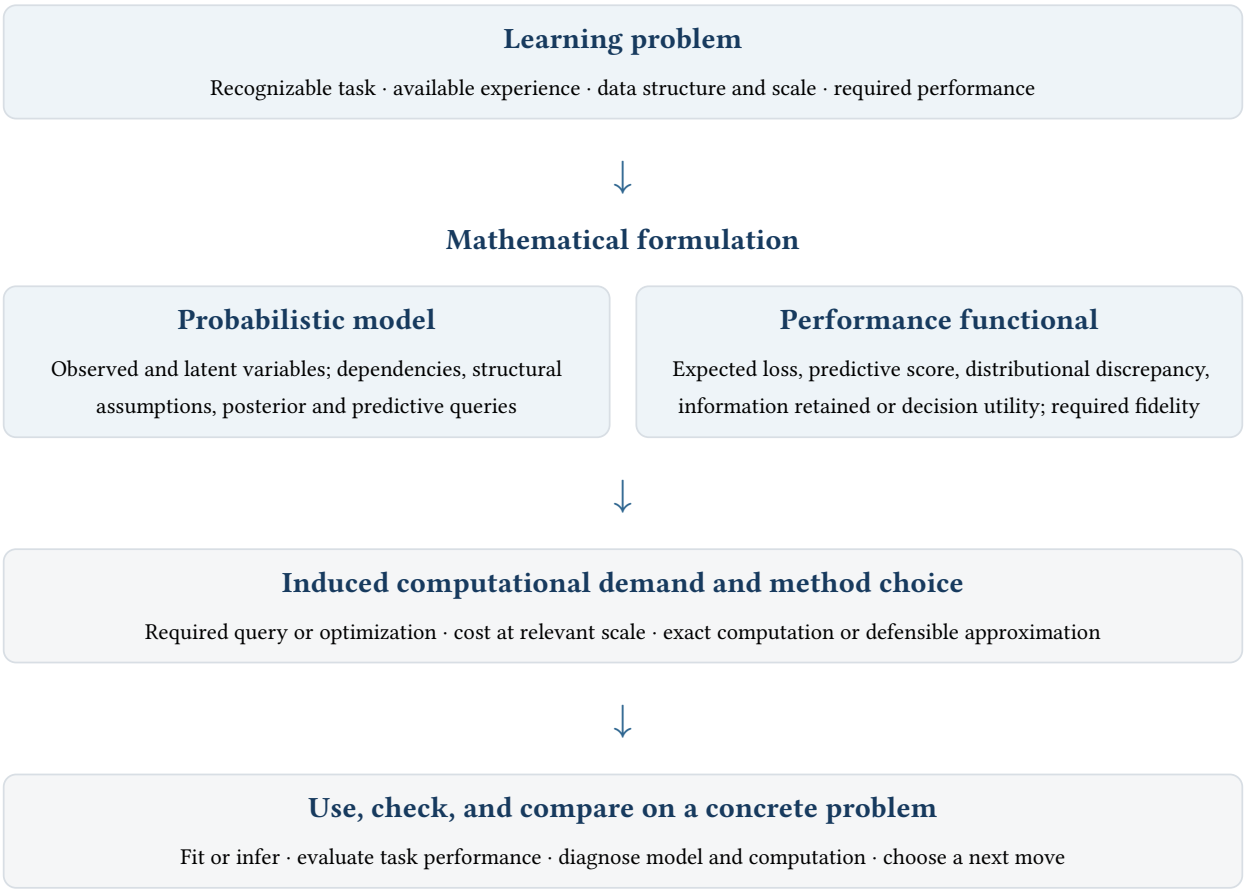
The mathematical formulation contains **two distinct, related objects**. A probabilistic model represents relevant observed and latent variables, their dependencies, and uncertainty. A performance functional specifies what counts as a good answer. For example, a predictive action a can be evaluated by

$$\mathcal{R}(a) = \int L(a, y) p(y) \, \mathrm{d}y.$$

The distribution under which actual performance is assessed need not equal the model we fit. Model mismatch, the choice of training surrogate, and numerical or approximation error are different possible sources of failure.

A formulation gives us **probabilistic queries**: conditional distributions, marginal probabilities, expectations, predictions or decisions, often requiring integrals or optimizations. L01 explicitly introduces conditioning, marginalization and summarizing a joint distribution. The rest of the course develops both richer probabilistic models and ways to perform the necessary computations at useful scale. A student's practical question is not merely "Which formula applies?" but "Which answer do I need, how can I compute it, and how can I tell whether it is good enough?"

The task–model–performance–computation relationship



A reasoning framework, not a rigid recipe: failures in application can lead us to revise the task, model, criterion or algorithm.

The course's main mathematical **moves** broaden **(i) what probabilistic models can express, (ii) what performance functionals can distinguish, and (iii) what computational methods can deliver**. The computational obstacle is **induced** by the data structure and size, the model and query, the performance requirement, and available resources. An algorithm becomes appropriate because it meets those particular demands, not because the syllabus has reached its chapter.

The ideal quantity and the practical calculation

A desired performance functional may be estimated by an empirical score or replaced for training by a surrogate objective. An ideal posterior or expectation may be approached using a point estimate, a Laplace or variational distribution, exact message passing, or a finite sample average. These substitutions are mathematically different; they must not all be called "posterior approximations." Some models make the relevant operation **exactly** tractable.

Every computational move should answer three linked questions: **Under what structural and data-scale conditions does it work? What computation does it save, compared with a plausible alternative? Is the resulting accuracy sufficient for the task we care about?** For example, matching local posterior curvature (L07) and optimizing a variational distribution (L09) can both yield a Gaussian proxy, but the former uses local

geometry and the latter fits a distributional objective. Either proxy may be sampled later. MCMC (L10) instead targets an integral through a dependent sample average. Exact sequence messages (L08) exploit conditional independence and need no posterior-family approximation. These distinctions must become practical judgments, not only definitions students can repeat.

How mathematical depth becomes practical familiarity

Each lecture needs a recognizable **problem encounter**: what is observed, what must be predicted, inferred, generated or decided, and how success will be checked. Its mathematical argument then explains a method's assumptions, what is computed and the regime in which the method pays off. Finally, students should see and use the approach on a concrete dataset, inspect its output and failures, and compare it with at least one meaningful baseline or alternative when feasible. This is a **design test**, not a mandatory slide sequence; some lectures develop representation or performance rather than a new algorithm.

A transparent toy model and a credible practical example have **different instructional jobs**. A small controlled case can expose a theorem, exact reference, bias or numerical failure that is otherwise hidden. A substantive case demonstrates why one would use the method and what it means to evaluate an answer. A lecture or notebook may use both, with much of the code or data preparation supplied, rather than forcing every student to implement two complete applications in an hour. Whenever possible, revisit an earlier problem with a new method: the difference between tools becomes visible when the question, dataset and performance criterion stay fixed.

The approximately two-hour lecture is responsible for the mathematical reasoning, conceptual comparisons and interpretation of representative results. The approximately one-hour notebook builds a bounded but real capability: **construct or run** a method, **read and validate** its output, and **judge** whether it answers the problem. Avoid notebooks that consist only of reproducing a plot or checking a derivative, but also avoid notebooks so elaborate that students spend the hour debugging scaffolding rather than understanding machine learning.

The role of Murphy and mathematical depth

Murphy's two books provide the main mathematical and conceptual source. Book 2 supplies the broader logic and methods; Book 1 often supplies more gradual derivations or useful introductory examples. The course is a deliberate selection, not comprehensive chapter coverage. Formal proofs and calculations are retained where they explain a method's ability, limits, or relation to other methods. Their pedagogical value is tested by the understanding and problem-solving judgment they make possible—not by the number of pages or results reproduced.

How the semester develops

What constitutes a class of learning problems?

A learning-problem family is not simply "medicine," "images," or a convenient dataset on which to demonstrate an algorithm. It is characterized by **the available experience and the structure and size of the data** (what is observed, missing, sequential, dependent, or high-dimensional), **the task and probabilistic question**, and **the performance requirement** (what must be estimated, predicted, preserved or decided, and how accurately). These induce the necessary calculations and their costs relative to a computational budget. **Computational difficulty is a consequence of the formulation, not a fourth independent ingredient.** Different datasets may share the same mathematical problem, while the same dataset may support several distinct questions.

But the purpose of this classification is **to help students solve recognizable machine-learning problems**. Every lecture therefore needs a practical destination as well as an intellectual contribution: students should

understand **what the problem looks like in data**, which method they would reach for, what its output means, how it is evaluated, and where it is likely to fail. The mathematical progression supplies a shared explanation for that repertoire. A lecture fails if students see only formulas without a usable technique, or only a technique without an intelligible reason for choosing it.

The semester follows a recurring argument: **recognize the learning problem → formulate its model and performance → identify the required computation and its scale → develop the mathematical move → use and evaluate the method**. This is a conceptual relationship, not a required presentation order. The notes and notebooks may begin with a practical puzzle or a mathematical result, whichever reveals the connection most clearly. We should cultivate both familiarity with individual methods and the ability to transfer that familiarity to a new dataset.

Representing observations and dependencies (L01–L03)

L01 — Ask different questions of the same observed system. When measurements are incomplete or noisy, the same dataset can support prediction of a missing measurement, inference of a hidden state, generation or a decision. The practical starting point is a multivariate measurement problem: students specify what they observed, what they want, and what counts as a useful answer; then use multivariate Gaussian conditioning to impute missing observations and quantify uncertainty. The **mathematical move** is to express different tasks as conditioning, marginalization and summarizing a joint distribution. The Gaussian case provides a complete, workable baseline; it also establishes the challenge of retaining this problem-solving ability when the relevant distributions stop being Gaussian. The delivered notebook gives initial practical experience with conditioning, rather than just a derivation of conditional moments.

L02 — Estimate and generate data with different kinds of outcomes. Binary measurements, real measurements, categories, counts and observed character transitions each call for an appropriate distribution. Students should recognize these data types, estimate a law from observed samples, score new cases and simulate additional examples. Exponential-family models make the connection between these tasks transparent: sufficient statistics summarize relevant data; derivatives of the log-partition function give moments; moment matching supports MLE; maximum entropy motivates distributions under stated constraints. The **practical exemplar** is a character bigram model: count transitions, normalize probabilities, score held-out names and generate names. The goal is not to reduce text modeling to bigrams, but to learn a complete small modeling cycle and recognize when the Markov/finite-statistic assumptions would be inadequate.

L03 — Represent and exploit structured dependence in many variables. An image, network or spatial measurement field is not well modeled as a collection of unrelated observations. Students should learn to recognize a plausible conditional-independence structure, specify its factors, and use a graphical model to predict or reconstruct missing components. DAGs, d-separation, Markov blankets and undirected potentials express different constraints on a joint probability distribution; the reduced representation may, but need not, make inference tractable. In the image-restoration exemplar, students construct an Ising prior and an observation likelihood, derive a single-site conditional, and perform a local reconstruction update. They must interpret the image and recognize that conditional optimization is not global MAP or a posterior marginal. The **transferable insight** is when a structural assumption enables modeling and which further computational question remains open for L08.

Performance, predictive likelihoods and incomplete data (L04–L06)

L04 — Judge distributions and representations, not just point predictions. A classifier can be correct but badly calibrated; a compressed representation can preserve some distinctions while losing others. These are

learning problems whose success cannot be described by a single accuracy or squared-error number. The **performance move** introduces entropy, cross-entropy, KL divergence and mutual information. Expected negative log score relates probabilistic predictions to KL discrepancy; mutual information describes a selected dependency, not universal usefulness for every decision. Students should compare **actual probabilistic predictions** and identify what different performance measures reward. The noisy parity-code example offers an exact small experiment in the tension between information, error probability and resource rate, but it is **not the practical destination of the lecture by itself**. It should be explicitly related back to evaluation of machine-learning predictions and learned representations. The notebook should make a real distinction between a metric calculated correctly and a model or decision that is useful.

L05 — Fit probabilistic predictions for outcomes that are not Gaussian. Predicting a species or a binary clinical outcome requires different observation distributions from predicting a continuous measurement; counts introduce another mean–variance relationship. GLMs extend the repertoire of conditional likelihoods, with links relating their parameters to predictors. For canonical links, their structured gradients and convex likelihoods (under appropriate conditions) make point estimation especially manageable, although full parameter-posterior inference may not be. The **computational move** is to use likelihood gradients, reverse-mode automatic differentiation and minibatch optimization for an MLE/MAP fit. Students fit and evaluate a softmax classifier on Iris, inspect its predicted probabilities and held-out score, and compare hand-derived gradients with those from software. The aim is familiarity with an end-to-end probabilistic classifier, not just a backpropagation identity. The model's predictive output, training objective and empirical performance must remain distinct.

L06 — Learn parameters when observations are incomplete or contain hidden labels. Suppose several physiological measurements are missing for some participants, or an observation belongs to an unknown mixture component. With unknown population parameters, imputing a value is no longer the whole task: we need to **fit the model using the observed data**. Marginal likelihood integrates over latent variables; filling each one in with a point guess changes the objective. Expectation–maximization exploits a tractable conditional posterior and manageable expected complete-data statistics to alternate inference with fitting. The ELBO/KL identity explains its likelihood-improvement guarantee under exact, improving steps. Gaussian incomplete-data fitting needs expected second moments **including the conditional covariance**, connecting L01′s Gaussian calculations to L02′s sufficient statistics. Practically, students fit parameters under controlled missingness, compare them with complete-case or mean-imputed baselines, and distinguish observed-data likelihood from reconstruction quality. The technique is worthwhile only if its E- and M-steps are tractable; EM supplies a point fit, not full parameter-posterior uncertainty.

Four ways to answer difficult posterior questions (L07–L10)

These lectures are **not four unrelated algorithms**, and they are not a staircase of increasingly advanced approximations. They offer different answers to overlapping problems: posterior summaries, predictions, hidden-state probabilities and decisions. Their value depends on the structure and scale of the data/model, the desired query, available resources and tolerable error. Students should learn the **specific mechanics** of each and compare which one they would use. In particular, L07 and L09 both build tractable posterior distributions; L08 exploits structure for **exact** marginalization; L10 estimates target integrals with dependent draws. A shared probabilistic prediction problem across L07, L09 and L10, together with controlled diagnostic examples, would make these differences visible without requiring three unrelated applications.

L07 — Use posterior geometry when an approximate Gaussian answer is sufficient (Laplace). In Bayesian logistic prediction, a fitted coefficient vector alone does not express parameter uncertainty or posterior-

averaged predictive probabilities. If the posterior is smooth, regular and strongly concentrated near a well-identified mode, Laplace constructs a Gaussian proxy from the mode and inverse Hessian. We can then compute or sample from its approximate marginals and predictions without the original posterior integrals. The computational payoff is strong when the parameter dimension and Hessian cost are moderate; neglected modes, boundaries, asymmetry and important tails may invalidate a downstream decision even if the fit looks good near its peak. Students should **use** the approximation to predict with uncertainty, not just compute a curvature. The one-parameter logistic example remains a transparent comparison with checked quadrature, but should be connected to an actual small, overlapping binary-classification problem and a point-prediction baseline.

L08 — Exploit local dependence for exact inference in a hidden-state sequence (message passing). Given noisy observations over time, a practical question is whether the underlying system changed state, when, and with what uncertainty. An HMM expresses conditional independence through local transitions and emissions. Filtering, smoothing, sequence evidence and the most probable whole path are distinct answers. Forward–backward dynamic programming rearranges sums over K^T possible histories into reusable local messages, taking $O(TK^2)$ work for dense finite-state transitions. This is exact for the stated model, unlike a posterior-family approximation. Students should infer and interpret state probabilities on a sequence and distinguish an uncertain boundary from a forced segmentation; a tiny enumerated case checks the code. A behavioral-state or sensor-sequence interpretation gives the algorithm a recognizable use, while the casino HMM remains an accessible mathematical testbed. The linear-Gaussian Kalman comparison shows another structural regime permitting exact filtering.

L09 — Fit an approximate posterior distribution by optimization (variational inference). When repeated posterior queries are costly, the Hessian is unwieldy, or a local Gaussian is not accurate enough for the quantities of interest, we can choose a tractable family of posterior approximations and **optimize a distributional objective**. The ELBO/reverse-KL principle fits the entire candidate distribution rather than matching only local mode and curvature. VI does not by definition require Gaussianity or unimodality. Mean-field or Gaussian approximations gain cheap computation but can lose correlations, tails or modes; flexible families may preserve more at greater cost. Students should be able to fit a simple variational distribution, compare uncertainty and predictions to a checked reference or Laplace baseline, and tell **family restriction** from **optimizer error**. The correlated-Gaussian test problem makes that diagnosis exact; a linked Bayesian prediction application should show how the method is actually used. Sampling to optimize the ELBO or from the fitted proxy is not what distinguishes VI from Laplace.

L10 — Estimate target expectations by sampling when a tractable proxy is inadequate (Monte Carlo and MCMC). Some posterior questions depend on correlations, rare regions or multiple modes, and an easily fitted Gaussian or variational proxy may miss what matters. If a target density can be evaluated up to normalization, MCMC constructs transitions whose invariant law is that target; sample averages approximate **specific** probabilities, risks and predictions. This does not impose a fixed posterior-family approximation, but finite-run mixing and autocorrelation can dominate the accuracy and computational cost. Metropolis–Hastings provides the core construction; Gibbs shows the special role of tractable conditionals. Students should **run, diagnose and interpret** a chain for a posterior query, compare estimates across chains and with a baseline, and explain why invariant transitions need not give good finite answers. The bimodal reference distribution exposes poor exploration; applying the same question to a posterior previously approached by Laplace or VI would provide practical comparative judgment.

Flexible prediction and learned latent representations (L11–L12)

These lectures demonstrate that extending the **model itself** can make the learning problem more realistic while making fitting or inference harder. A usable neural method must earn its complexity through predictive or generative performance and practical computational feasibility. The mathematical thread—differentiable functions and amortized approximate inference—supports the practical work of fitting and checking expressive models.

L11 — Learn nonlinear conditional probability distributions (probabilistic neural networks). For many datasets, an affine GLM or constant-variance response model cannot express the shape or conditional variability of observations. A neural network can parameterize a likelihood through nonlinear functions, such as a Gaussian with input-dependent mean and scale. Reverse-mode autodiff and minibatch training still work for the composed computation, although the optimization is generally nonconvex. Students should train a small conditional model, compare it against a meaningful fixed-scale or simpler baseline on held-out cases, and interpret log score, mean error and predictive intervals. A credible analysis distinguishes observation noise conditional on fitted weights from uncertainty about the weights: a fitted scale is not a Bayesian posterior. The synthetic heteroskedastic test case exposes known truth; its role is to teach whether added expressiveness helps the practical prediction task, not simply that a network can fit an arbitrary curve.

L12 — Generate and represent high-dimensional observations through amortized latent inference (VAEs). An unlabeled collection of high-dimensional observations creates different tasks: generating new plausible cases, reconstructing observed cases, and learning compact representations. A flexible latent generative model $p(z)p_{\theta(x|z)}$ can model such data, but separate posterior inference for every new observation is costly at scale. A shared encoder $q_{\varphi(z|x)}$ **amortizes** VI: training is expensive, but applying the inference map to many related cases can be cheap. The reconstruction/rate tradeoff reflects the required task performance; average encoder-to-prior KL is **not** identical to mutual information. Students should train or complete a compact VAE, generate from the prior, reconstruct actual examples, inspect latent codes and compare at least two fitted tradeoffs on meaningful measurements. They should also recognize posterior collapse, inference-family or amortization error, and nonidentifiability: an interpretable plot is not evidence of recovered causal factors. This is a concrete generative-modeling skill and a synthesis of L04, L06, L09 and L11, not merely another application of the ELBO.

L13 — Demonstrate practical selection and justification on a new problem. Student presentations should begin with a recognizable dataset and problem, identify the task and performance requirement, select a plausible probabilistic model and method, and show an actual result with a meaningful check or comparison. Mathematical explanations should clarify what their algorithms deliver and what remains uncertain. The course's success is visible when students can use a concrete method **and** give a principled reason for choosing or rejecting it—not when they can only recite the unified architecture or reproduce a worked notebook.

Proposed lecture and problem-solving practice map

The thirteen-meeting plan is a **revisable course proposal**, not a set of commitments protected from educational criticism. L01 has been delivered, L02 has an agreed design, and L03 and the earlier L04 coding split have prior instructor decisions that must be recorded when proposals change. Those historical facts are not reasons to retain a weaker future plan. Existing lecture-scope files, review records, source readings and notebook references remain evidence to reconcile before implementation; a change here is **not** an executed or instructor-approved replacement of those artifacts.

Each row connects an identifiable learning problem, a mathematical exemplar, and **bounded hands-on competence**. One notebook should generally provide meaningful use and evaluation of a method on data; when the chosen derivation needs a tiny exactly checkable example, supplied code or reference results can keep that diagnostic alongside the application. L07, L09 and L10 particularly benefit from revisiting a shared Bayesian prediction problem so that students compare methods, not just benchmark unrelated algorithms. Practical datasets, scaffolding, runtime and the one-hour limit still require separate feasibility checks.

No.	Lecture	Topics covered	Book 2 — primary	Book 1 — supplemental	Student problem, method use, evaluation and references
1	**Probabilistic ML and Gaussian systems** Delivered	Machine-learning task framing; probabilistic models; Bayes review; covariance and correlation; multivariate Gaussians; conditioning; missing-data inference; linear Gaussian systems.	• 2.1.5–2.1.6 Conditional probability; Bayes • 2.2.1.1 Bernoulli and binomial • 2.2.4.1 Beta distribution • 2.3.1.1 Gaussian joint distribution • 2.3.1.3–2.3.1.5 Gaussian conditionals; information form • 2.3.2.1–2.3.2.2 Linear-Gaussian systems • 3.2.1 Coin-flipping Bayes • 3.4–3.4.1 Conjugate priors; binomial	• 1.1–1.5 ML tasks and datasets • 2.3 Bayes' rule • 2.4.1 Bernoulli; binomial • 2.7.4 Beta distribution • 3.1.1–3.1.3 Covariance; correlation • 3.2.1–3.3.4 Multivariate and linear Gaussians • 4.3–4.3.1 Empirical risk • 4.6.1–4.6.2 Beta-binomial Bayes	**Gaussian measurement imputation (already delivered).** Condition a multivariate Gaussian on observed measurements, infer missing values and their uncertainty. Reference: Existing L1 solved notebook.
2	**Exponential families and learning from sufficient statistics** Earlier agreed design	Exponential-family likelihoods, sufficient statistics and moment matching; infer and evaluate a categorical transition model. Describe data type, probability law and the consequences of the Markov assumption.	• 2.2.1.2 Categorical distribution • 2.4.1–2.4.2 Exponential-family form; examples • 2.4.3–2.4.4 Log partition; moments • 2.4.5 MLE; moment matching • 2.4.7 Maximum entropy • 2.6.1–2.6.3 Markov chains; language models		**Character bigram prediction and generation.** Count transitions in baby names, fit a categorical transition matrix, compute held-out name log scores and generate plausible examples. Compare with an independent-character baseline and examine a failed or implausible output. **Competence:** Fit, score, generate and critique a simple distributional model. Reference notebook.
3	**Graphical models: factorization and conditional independence** Earlier agreed scope; proposed revision	Joint modeling of many dependent variables: DAG factorization, conditional independence, Markov blankets, undirected potentials and the Ising model. Use structural assumptions to derive a practical reconstruction calculation, while distinguishing it from exact marginal inference.	• 4.2.1–4.2.2 DAG factorization; student network • 4.2.4 / selected 4.2.8 Independence; plates • 4.3.1–4.3.2.1 / 4.3.6.1 Undirected models; Ising; separation • 4.5.1 Directed/undirected independence comparison • 4.3.8 Image restoration: notebook	• Optional 3.6.1–3.6.2 Graphical-model orientation	**Probabilistic image restoration.** Construct a locally coupled Ising prior and noisy-image likelihood; derive a site conditional and implement a scaffolded local reconstruction update. Compare with independent pixel restoration, assess visible errors and diagnose local optima. **Competence:** Recognize when local factors help construct and use a structured model, without confusing local MAP updates with posterior marginal probabilities. Reference notebook.
4	**Information theory: distributional comparison and information preservation** Revisable proposal	Proper probabilistic scoring, entropy, cross-entropy, KL divergence, mutual information and data processing. Judge predictive distributions and representations in terms of the task's performance, not accuracy alone. A small noisy-code construction illustrates information versus error and rate.	• 5.1.3 / 5.1.6; 5.2.1 / 5.2.4 Information and uncertainty • 5.3.1–5.3.4 Shared information and processing • 5.4.1 Code-length bounds: statement only • 5.5 Noisy-channel example: three-bit parity model	• Selected 6.1 / 6.2.1–6.2.2 / 6.3 Uncertainty and shared information	**Compare probabilistic predictions and information-preserving representations.** Given prepared predictions and labels for a recognizable classification dataset, compare accuracy, held-out log score and a simple reliability check; examine a discrete feature representation and calculate what label information it retains. Use a small finite channel/parity example as an optional exact check of error versus information and resource rate. **Competence:** Choose and interpret performance measures; explain why the best classifier by one metric may not be best for another task. Coding reference; supplementary. **Proposal change:** The previously selected five-bit correcting-code notebook is no longer the default one-hour exercise; feasibility and source coverage require review.
5	**Generalized linear models, automatic differentiation and SGD** Revisable proposal	Gaussian, Bernoulli, Poisson and categorical conditional prediction; GLM links, canonical gradients and convexity when applicable. MLE/MAP point fitting via autodiff and mini-batch SGD. Use predictive probabilities and held-out scores to judge fitted classifiers.	• 15.1 / 15.1.1 / 15.1.3 Common model structure and derivative • 15.1.2 Brief alternative-link comparison • 15.3.2 Three-class probabilities	• Selected 7.8.2 / 7.8.5 Matrix derivatives • Selected 8.2 / 8.4.1 / 8.4.3 Gradient and batch-update support	**Iris probabilistic classification.** Fit a three-class softmax model with prepared PyTorch scaffolding. Check one manually derived gradient against autograd, train from a supplied baseline, inspect confusion and predicted probabilities, and compare held-out log score with a simple alternative. **Competence:** Set up, fit, interpret and

No.	Lecture	Topics covered	Book 2 — primary	Book 1 — supplemental	Student problem, method use, evaluation and references
			• Selected 6.2; 6.3.1–6.3.2 Automatic differentiation and batch updates		diagnose a probabilistic classifier, rather than merely verify backpropagation. Reference notebook. **Limit:** First encounter with PyTorch; supply data splits and helpers.
6	**Latent-variable learning and expectation-maximization** Revisable proposal	Latent-variable joint likelihood and observed-data marginal likelihood; ELBO/KL identity, exact EM conditional expectations and monotonic improvement. Use conditional second moments for missing-data Gaussian parameter fitting; recognize mixture responsibilities and missingness assumptions.	• Selected 6.5.1 / 6.5.3 Probability bound and EM • 6.5.4 Gaussian missing-data updates • 3.11 When the missing-data mechanism can be ignored • 28.2.1 Group-assignment probabilities: brief example	• Selected 3.5 / 8.7 Grouped observations and EM explanation	**Fit a population model with incomplete measurements.** Work with a prepared multivariate measurement dataset with controlled missingness. Assemble expected sufficient statistics, including conditional covariance, and run EM in a supplied driver. Compare against a complete-case or mean-imputation baseline; assess observed-data log likelihood and held-out reconstruction separately. **Competence:** Fit, evaluate and diagnose an incomplete-data model and recognize when EM is or is not an appropriate tool. Reference notebook.
7	**Bayesian computation: grid reference and Laplace approximation** Revisable proposal	Bayesian logistic-regression predictions and parameter uncertainty; MAP versus posterior queries; Laplace mode/Hessian construction, conditions for local Gaussian validity, computational scaling, and query-specific error.	• 7.1–7.3 What posterior questions require: short introduction • Selected 7.4.1–7.4.3 Most likely parameter, grid and Gaussian shortcut • Selected 7.5 Checking approximation quality • Selected 15.3.5–15.3.6 One-parameter logistic example		**Bayesian binary classification, first inference route.** Use a prepared, overlapping two-class subset of the Iris task, with few parameters, to compare point predictions with posterior-averaged predictions from a Laplace Gaussian. A one-parameter restriction and checked numerical grid provide an exactness **diagnostic**, not the practical motivation. Inspect uncertainty and when local curvature is misleading. **Competence:** Construct and use a Laplace posterior proxy and evaluate the downstream prediction. Reference for numerical check. **Feasibility:** Supply fitting and Hessian helpers; do not require two complete independent fits.
8	**Exact inference in structured sequence models** Revisable proposal	HMM filtering, smoothing, evidence and path MAP as different answers to a hidden-state problem. Forward–backward factorization and exact dynamic programming; scaling, computational cost, and brief linear-Gaussian Kalman comparison.	• 9.2.1–9.2.5 Hidden-state calculations and scaling • 9.2.6 Whole history versus separate state decisions • Selected 8.1.2 / 8.2.2 Gaussian filtering: short application		**Infer hidden behavioral or instrument states from a noisy sequence.** Use a prepared labeled-context sequence and a small HMM: compute filtered/smoothed probabilities, identify uncertain transitions and compare a state-by-state prediction with path MAP or a naive baseline. Check a tiny enumerated sequence for correctness. **Competence:** Use exact message passing and interpret probabilistic state inference instead of merely producing a sequence of hard labels. Casino HMM diagnostic reference. **Feasibility:** Prepare the sequence and model parameters; no full state-space-model fitting required.
9	**Variational inference: posterior geometry and pathwise gradients** Revisable proposal	Variational distributional optimization via reverse KL/ELBO, approximation-family and optimizer error, pathwise gradients, and mean-field limitations. Contrast the fitted Gaussian with Laplace on the same kind of posterior query.	• 10.1.1–10.1.2 Fitting a restricted distribution • 6.3.3 / 6.3.5; 10.2.1.1 Derivatives through Gaussian draws • 6.3.4 Another gradient estimator: statement only • 10.3.1 Coordinate updates; 10.1.5 learned inference: introduction only		**Bayesian binary classification, second inference route.** With prepared likelihood and optimizer code, fit a diagonal-Gaussian variational posterior for the overlapping binary Iris model used in L07; compare task-relevant predictions and uncertainty with Laplace and a supplied reference. A correlated Gaussian with known answer isolates mean-field family error from optimization error. **Competence:** Fit and diagnose a variational posterior and decide whether lost dependence matters for the prediction. Reference VI implementation. **Feasibility:** Limit student code to the defining objective, parameters, and interpretive comparisons.

No.	Lecture	Topics covered	Book 2 — primary	Book 1 — supplemental	Student problem, method use, evaluation and references
10	**Monte Carlo and MCMC: invariance, mixing and estimator error** Revisable proposal	Monte Carlo estimators for posterior queries; MCMC target invariance versus mixing, Metropolis–Hastings correction, Gibbs full-conditionals, finite-run error and query-dependent effective sample size.	• 11.2–11.2.2 Averages and sampling error • 2.6.4; 12.2.1–12.2.2 Invariant distribution and acceptance rule • 12.3.1–12.3.2 Conditional-coordinate sampling: brief derivation • Selected 12.6.3 Dependent-sample error; 12.5.3 Hamiltonian mechanics only		**Bayesian inference by sampling, third route.** With an MH driver and precomputed L07/L09 comparisons, estimate posterior expectations and predictive probabilities for the same small binary-classification model; examine run-to-run variability and query-specific error. A supplied bimodal target is a stress test for mixing and mode exploration. **Competence:** Run and diagnose MCMC estimates rather than equate stationarity with successful finite computation. Bimodal diagnostics reference. **Feasibility:** Supply proposal, plotting and diagnostics so one hour is enough.
11	**Neural conditional models: heteroskedastic likelihood and regularization** Revisable proposal	Nonlinear differentiable conditional models with learned mean and positive scale, heteroskedastic Gaussian log score, SGD/AD reuse, regularized point fitting and differences between conditional noise and weight uncertainty.	• Selected 16.2.1–16.2.2 / 16.3.1 Small feedforward networks • 15.2.2.1 Gaussian prior and weight shrinkage • 14.2.1 Evaluating predicted probabilities	• 13.2.4.4 Noise that depends on input • 13.5.2 Weight shrinkage; 13.5.1 stopping training: brief mention	**Conditional probability beyond a GLM.** Train a small mean/scale neural model on a prepared nonlinear dataset and evaluate against fixed-scale and simpler-function baselines. Compare held-out log score, mean error, and regional interval behavior, not just training loss. **Competence:** Assess whether a flexible predictive distribution earns its complexity and distinguish observation variability from parameter uncertainty. Reference.
12	**Variational autoencoders: amortized inference and rate–distortion** Revisable proposal	High-dimensional latent generative model and per-case approximate posterior; shared amortized encoder, neural decoder, ELBO and pathwise training, reconstruction/rate tradeoff, posterior collapse and representation identifiability.	• Selected 21.1–21.2 Model, inference network and training objective • 21.3.1 Changing the penalty weight • Selected 5.4.2 Reconstruction/information tradeoff • Selected 21.4 When the model ignores hidden coordinates		**Generate and represent mobility measurements.** Fit or complete a compact VAE on supplied 19-feature observations. Inspect separate reconstructions and **prior-generated** cases, compare tradeoffs across supplied settings, and look at latent descriptions without claiming causal recovery. **Competence:** Use and critically evaluate a generative latent model, including whether the encoder and codes are useful. Mobility-data provenance. **Feasibility:** Require verified data access, prepared fit baselines, and a CPU/runtime pilot before fixing the exercise.
13	**Project presentations and course synthesis**	Student project presentations connecting probabilistic assumptions, computation, prediction, evaluation, uncertainty and scientific interpretation. No additional substantive lecture topic is committed at this stage.			No notebook — project presentations and synthesis.

Selection, omissions, and what students actually gain

This course develops a usable and intellectually connected repertoire, not an exhaustive survey of Murphy. The choices below trade some topical breadth for repeated **method use**, **comparison of output with a task-specific criterion**, and **diagnosis of failure**. They are current design hypotheses rather than completed student trials. The architecture audit records evidence and feasibility concerns; the current semester's lecture scopes and notebook drivers must be reconciled before declaring new assignments final.

Other possible topic	Choice in this plan	What we give up
More advanced optimization	Basic derivatives and small-batch updates are sufficient here	No separate study of curved-geometry updates, expensive black-box search, transport or subset selection.
Particle methods for changing hidden states	Exact sequence calculations in L08 and sampling theory in L10	No method for tracking a complex changing state with a cloud of weighted samples (Book 2 Ch. 13).
MRI classification and continuous forecasting	Missing-data fitting and discrete hidden-state lab selected	Less biomedical and forecasting practice. Either application can replace its corresponding lab after review.
More complex regression and prediction checks	Common models in L05 and one neural probability model in L11	No full sparse/hierarchical regression, calibration or guaranteed prediction-interval course.
Larger inference toolkits	One checked method per main lab	No general-purpose graph solver, broad variational-family catalogue or full Hamiltonian sampler derivation.
Bayesian neural networks and Gaussian processes	Small fitted networks only	No separate treatment of uncertainty about network weights or Gaussian-process prediction.
Other generative models	A variational autoencoder in L12	No flow, diffusion or adversarial model teaching. These are different ways to learn data distributions and need their own time.
A complete algorithm catalogue	One working exemplar per lecture, with recurring dataset comparisons where they clarify choices	Less survey breadth; more ability to explain, execute and diagnose selected techniques.
Every method on a different isolated toy problem	A shared Bayesian classifier in L07/L09/L10 plus small transparent diagnostics	Less variety in those notebooks, but a more meaningful comparison of posterior approximations and sampling at fixed task and performance.